

Email Header Analyzer

with Independent Verification and Threat Intelligence

A tool that turns a raw email header into an evidence-backed, explainable verdict

M. Ismail

SOC Internee, Ebryx

Reviewer: Syed Jawad

Technical Whitepaper

July 23, 2026

Contents

1	Abstract	2
2	Problem Statement	2
3	Architecture Overview	2
3.1	Module boundaries	2
4	Parsing Pipeline	3
5	Independent Verification	3
5.1	What "asserted" vs. "verified" means on screen	3
5.2	Correctness bugs this differentiator actually caught	4
6	Indicators of Compromise and Threat-Intelligence Enrichment	5
6.1	Off by default, on purpose	5
6.2	Providers	5
6.3	Verified against live providers	6
7	Risk Engine and Verdict	6
7.1	Correlate, don't override	6
7.2	What the verdict deliberately cannot say	7
8	Evidence-Backed Verdict: Worked Example	7
9	Interface	7
10	Design Decisions and Deviations from the Brief	8
11	Threat Model and Limitations	9
11.1	What this tool is not	9
11.2	Trust boundary	9
11.3	Data handling	9
12	Testing and Quality	9
13	Reference Repositories	10
14	Setup and Deployment	10
15	Conclusion	10

1 Abstract

Email header analysis is a core SOC triage skill, but most open tooling stops at *parsing*: it displays what a header says without checking whether any of it is true. This project takes a raw email header, reconstructs the delivery path, **independently re-evaluates** SPF, DKIM and DMARC against live DNS rather than trusting what the receiving server recorded, extracts indicators of compromise, enriches them against real threat-intelligence APIs, and produces a verdict where every contributing claim is traceable to a specific header line. The design goal, stated plainly in the project's own planning document: *every claim the tool makes on screen must be one the analyst can justify from the evidence shown next to it*. No opaque scores, no fabricated intelligence, no verdict without its reasons.

This document describes the architecture, the independent-verification approach that differentiates it from parse-only tools, the risk-scoring model, the threat-intelligence integration, and the engineering decisions behind each — including the deliberate deviations from the original build brief and why each was made.

2 Problem Statement

A SOC analyst receiving a reported phishing email is handed a raw header and asked: is this real? The manual answer requires several independent checks — read the **Received:** chain bottom-up, compare **From/Reply-To/Return-Path** organisations, re-derive SPF and DKIM against current DNS (because a header's recorded **Authentication-Results** can simply be forged — RFC 8601 §7.1 says so explicitly), check DMARC alignment and policy, and cross-reference any IPs, domains or URLs against threat intelligence. Existing open tools reviewed for this project (see Section 13) automate the parsing step but not the verification step: they display whatever the header claims, including claims an attacker fully controls.

The tool built here automates the full manual workflow — not just the parsing half of it — so its output is something an analyst can put directly into a ticket, with the reasoning attached.

3 Architecture Overview

The application is a server-rendered FastAPI service. There is deliberately no database, no queue and no cache server — analysis is synchronous, reports live in a bounded in-memory TTL cache, and nothing outlives the process. This is a considered choice, not an omission: a reference project reviewed during planning shipped PostgreSQL, Redis and Celery and used none of them meaningfully (Section 13).

3.1 Module boundaries

- **app/core/** — pure analysis logic: header parsing, **Received:** chain reconstruction, authentication parsing, identity comparison, IOC extraction, domain analysis (lookalike/homoglyph detection), the risk engine. No I/O.
- **app/core/verification/** — independent SPF/DKIM/DMARC/DNSBL/FCrDNS evaluation. The only place live DNS is queried.
- **app/integrations/** — one file per threat-intel provider, each behind a common **Provider** interface.

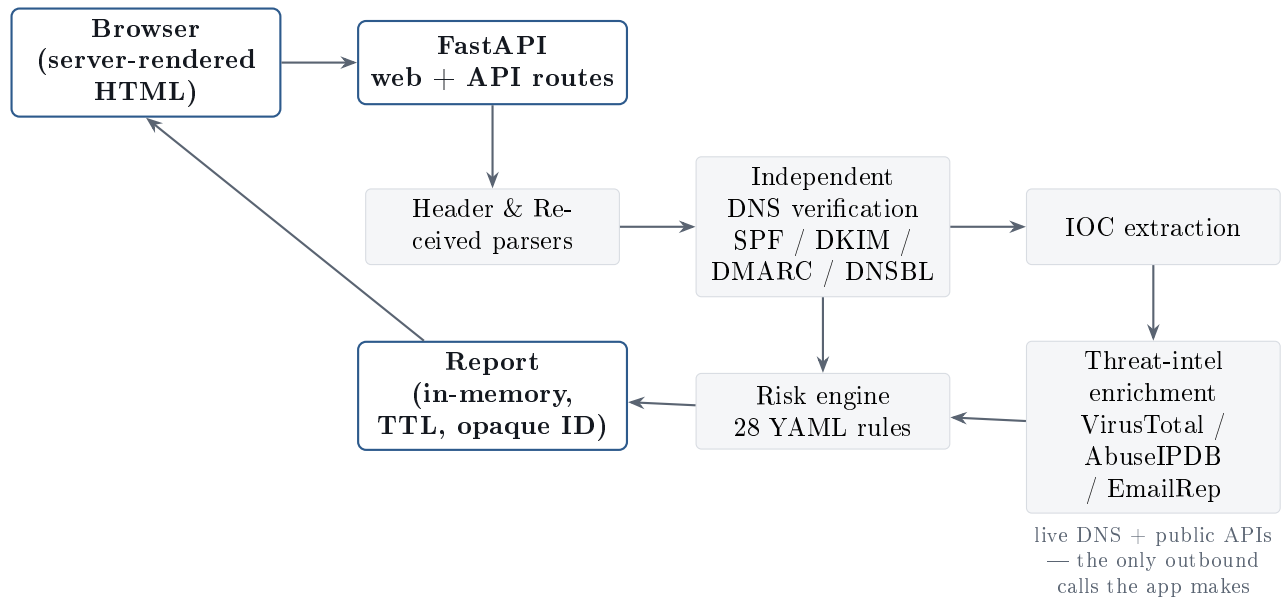


Figure 1: Request lifecycle. Live outbound network calls happen in exactly two places: DNS resolution during verification, and threat-intel HTTP calls during enrichment (opt-in, off by default).

- `app/services/` — orchestration: `analysis_service` ties parsing, verification, enrichment and risk scoring into one report; `enrichment_service` owns concurrency limits, caching and demo-mode fixtures; `report_service` owns JSON/Markdown/PDF export.
- `app/routes/` — HTTP surface: server-rendered pages (`web.py`), a typed JSON API (`api.py`), report export endpoints.

4 Parsing Pipeline

The `Received:` parser is written from scratch. The best available open `Received:` parser (`eml_parser`, GOVCERT-LU) was evaluated and rejected: it is AGPL-3.0, and §13 of that licence triggers on any network-served use of the work — exactly what this application is. Reimplementing it clean-room was the only option compatible with an MIT-licensed, publishable project.

5 Independent Verification

This is the project’s primary differentiator. Every one of the six reference open-source tools audited during planning (Section 13) is parse-only: they read whatever `Authentication-Results` the receiving server wrote and display it as-is. RFC 8601 §7.1 is explicit that this header is forgeable — an attacker can place `Authentication-Results: yourcompany.com; spf=pass; dkim=pass; dmarc=pass` into a message they compose themselves, and a tool that reads it uncritically shows a green pass.

5.1 What "asserted" vs. "verified" means on screen

Every authentication row in the results UI shows both what was *asserted* (what the receiving infrastructure claimed, with a trust level — UNKNOWN unless the deployment explicitly configures `TRUSTED_RECEIVER_DOMAINS`) and what was *independently verified* (what this tool itself derived

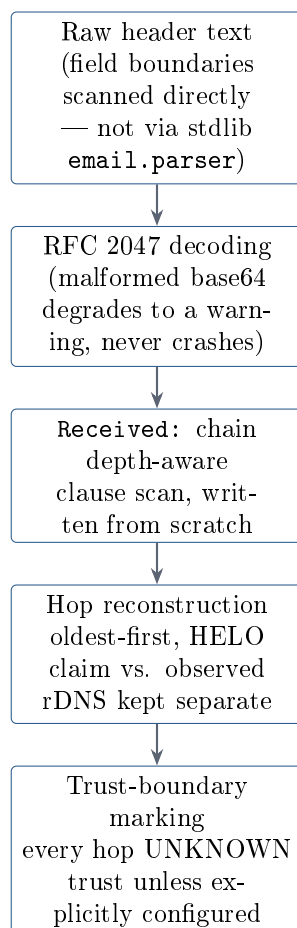


Figure 2: Header parsing preserves raw bytes, original field order and duplicate headers — information a `stdlib-email`-based parser silently discards.

from live DNS, right now). The two can disagree — that disagreement is itself surfaced as evidence, not hidden.

5.2 Correctness bugs this differentiator actually caught

Three real defects were found by the project's own test suite while building this layer, each worth stating because they show why "verify it yourself" is harder than it sounds:

1. `email.header.decode_header` raises `HeaderParseError`, which is *not* a `ValueError` subclass — a malformed base64 encoded-word escaped the exception handler entirely before this was caught and degraded to a warning.
2. `.example` is not in the Public Suffix List. `tlsextract` returned no suffix for it, so `mail.bank.example` and `bank.example` compared as *unrelated* organisations — silently defeating the impersonation check on every test fixture. Fixed with a documented last-two-labels fallback.
3. `pyspf` dispatches DNS through a module-level resolver and treats an empty cache entry as a cache miss. Seeding its cache for offline tests did not actually prevent it — the test suite was silently performing live DNS lookups. In production this meant every `include:/a:/redirect=` directive bypassed the configured resolver and timeout entirely. Fixed by patching the resolver dispatch point directly.

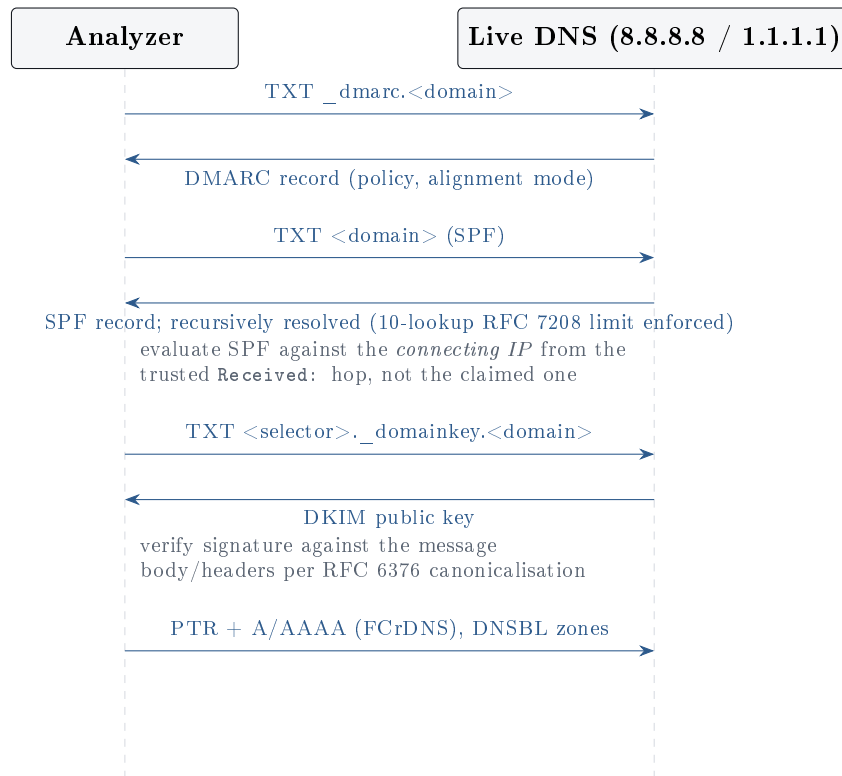


Figure 3: Independent verification queries live DNS itself rather than trusting the recorded **Authentication-Results:** header. Results are shown *asserted vs. verified*, side by side, on the results page.

6 Indicators of Compromise and Threat-Intelligence Enrichment

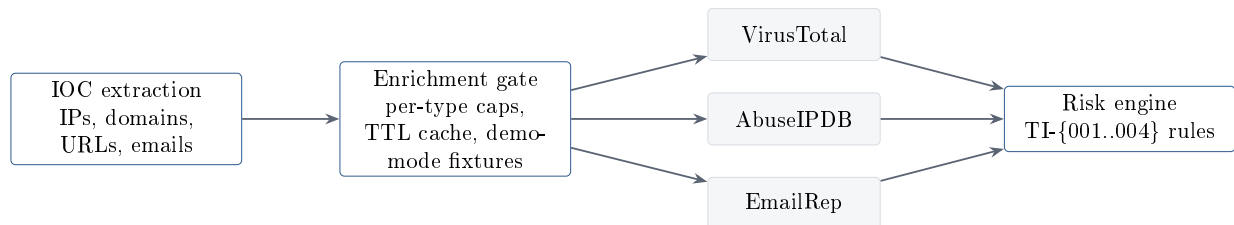


Figure 4: Enrichment is a single deliberate gate (`enrichment_service.py`): it, and nothing else, decides whether a provider is ever called. Providers themselves never know about caching, per-type limits or demo mode.

6.1 Off by default, on purpose

`ENRICHMENT_ENABLED` defaults to **false**. This is deliberate, not a missing feature: enabling it sends extracted indicators to third parties, and phishing URLs are frequently unique per recipient — submitting one to a public scanner can itself signal to the attacker that their campaign was detected. Turning enrichment on is an explicit operator decision, never an inherited default.

6.2 Providers

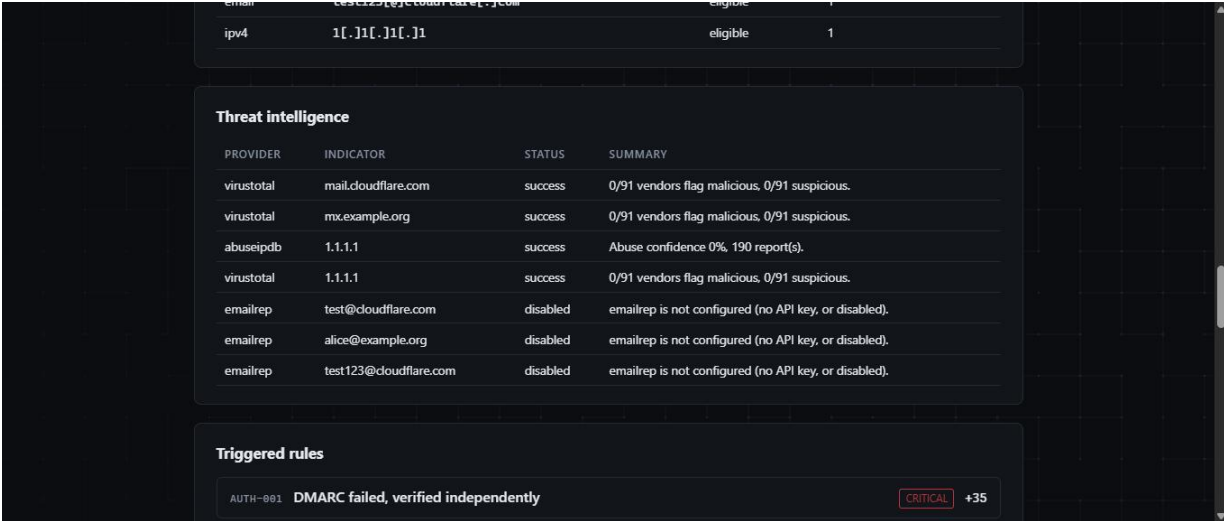
- **VirusTotal** — IPs, domains and URLs, checked against up to 90+ vendor engines.

- **AbuseIPDB** — IP abuse-report history and confidence score. Country code is captured for display only; it is asserted by test (`test_country_never_contributes_to_score`) that it never influences the risk score.
- **EmailRep** — sender-address reputation, feeding directly into the "why this verdict" evidence for BEC/impersonation findings.

Every lookup — including a **DISABLED** one — is recorded and shown. The UI never silently omits a check that wasn't run; it says so explicitly, because "not checked" and "checked and clean" are different findings and conflating them is a false sense of safety.

6.3 Verified against live providers

The threat-intelligence path in Figure 4 was exercised end-to-end against the real APIs (not mocked) as part of this project's own verification: Figure 5 shows the actual response.



PROVIDER	INDICATOR	STATUS	SUMMARY
virustotal	mail.cloudflare.com	success	0/91 vendors flag malicious, 0/91 suspicious.
virustotal	mx.example.org	success	0/91 vendors flag malicious, 0/91 suspicious.
abuseipdb	1.1.1.1	success	Abuse confidence 0%, 190 report(s).
virustotal	1.1.1.1	success	0/91 vendors flag malicious, 0/91 suspicious.
emailrep	test@cloudflare.com	disabled	emailrep is not configured (no API key, or disabled).
emailrep	alice@example.org	disabled	emailrep is not configured (no API key, or disabled).
emailrep	test123@cloudflare.com	disabled	emailrep is not configured (no API key, or disabled).

Triggered rules	
AUTH-001	DMARC failed, verified independently CRITICAL +35

Figure 5: Live enrichment results for a test message — real VirusTotal and AbuseIPDB API responses, not fixtures. 1.1.1.1 returns 0/91 vendors flagging malicious from VirusTotal and 0% abuse confidence with 190 historical reports from AbuseIPDB. EmailRep shows **disabled** here because no API key was configured for that provider at capture time — exactly the "not checked, not clean" distinction described above.

7 Risk Engine and Verdict

Risk rules are declared in YAML (`app/core/rules/rules.yaml`), not embedded in Python — 28 rules at time of writing, each with a stable ID (e.g. AUTH-001, IDN-002, RTE-004, TI-004), an evidence strength, a score contribution, and, critically, a *possible legitimate explanation* and a *recommended action*. This lets the verdict card name exactly which rules fired, in plain language, reviewable by someone who has never read the Python.

7.1 Correlate, don't override

The engine deliberately has no single rule that can unilaterally declare a verdict. Evidence accumulates; the final verdict is a function of the accumulated score and the confidence in that score,

not any one rule short-circuiting the rest. The exception is explicit and visible: if a *matched pattern* (a named, high-confidence combination) fires, it is surfaced as `risk.matched_pattern` on the verdict — but it still contributes through the same scoring path, not as a bypass.

7.2 What the verdict deliberately cannot say

There is no `SAFE` verdict and no `BEC_CONFIRMED` verdict. Header evidence cannot establish that a message is harmless — a compromised but genuine mailbox passes every authentication check there is — and it cannot establish business context, which an actual BEC determination requires. The five possible verdicts are *Likely Legitimate (based on available header evidence)*, *Suspicious*, *Likely Phishing*, *Possible BEC/Impersonation*, and *Inconclusive*. Every one of them is a statement about the evidence, not a claim about ground truth.

8 Evidence-Backed Verdict: Worked Example

Figure 6 shows the bundled phishing sample run through the tool with live threat-intel enrichment on. The verdict card states a score and a confidence level; immediately below it, the *Why this verdict* panel lists the specific evidence, each citation naming the exact header field and value that produced it — not a generic "this looks suspicious," but "Reply-To is support@secure-verify-alerts.example, but From is in northwind-secure.example. Replies leave the sender's organisation," with a link to the raw header line it was read from.

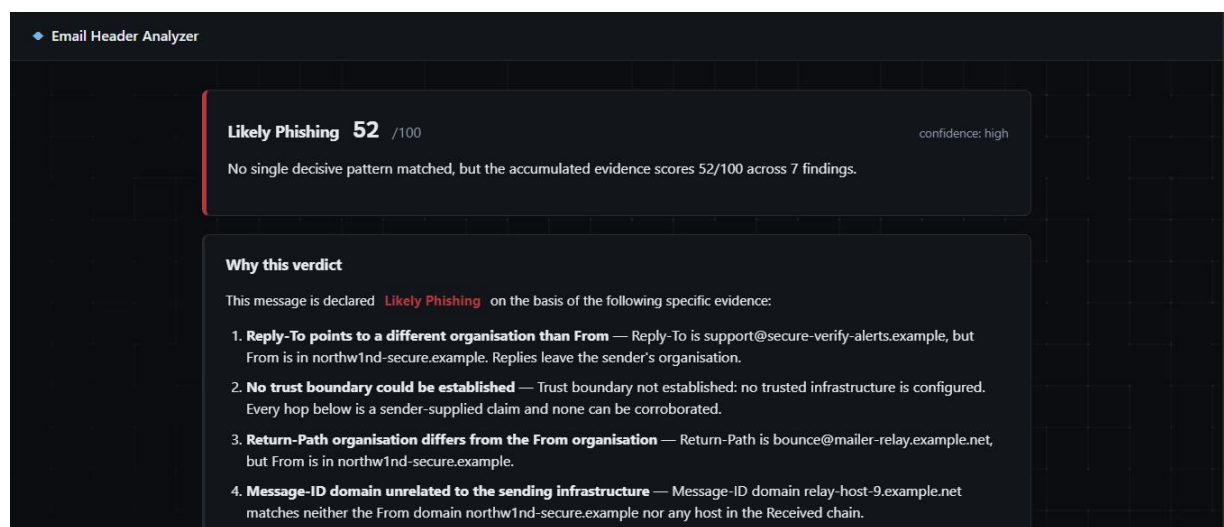


Figure 6: Evidence-backed verdict for the bundled phishing sample. Every numbered citation traces to a specific header field; the full findings list below it (not shown) adds, for each finding, why it matters, a possible legitimate explanation, and a recommended action.

9 Interface

Server-rendered Jinja2 templates, not a client-side framework — a deliberate deviation from the original brief's Bootstrap/React-leaning suggestion. Two reasons: Jinja2 autoescaping is the primary XSS defence against input that is, by definition, attacker-controlled (a raw email header); and a server-rendered app with no build step and no CDN dependency works air-gapped, which several reference tools do not.

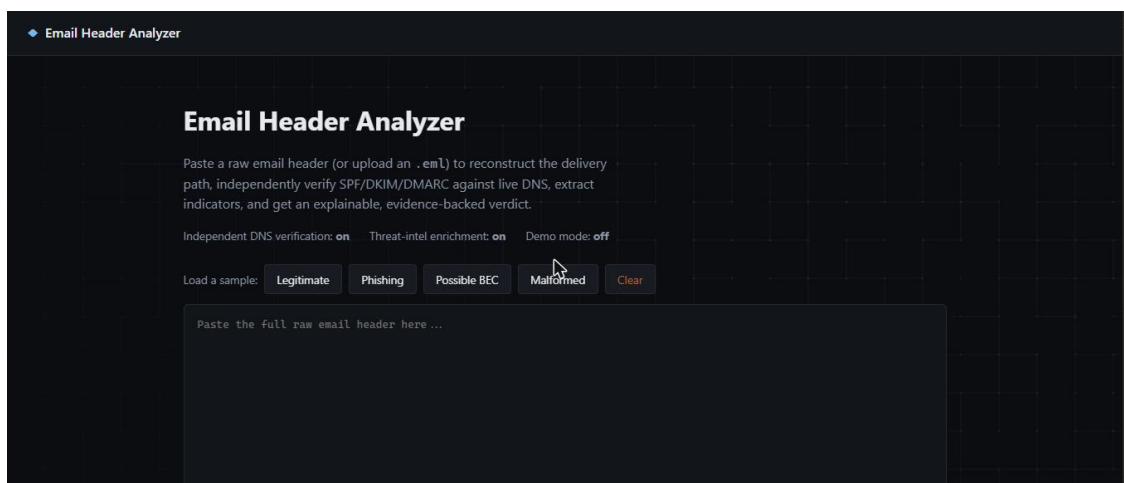


Figure 7: Landing page. Configuration state (DNS verification, threat-intel enrichment, demo mode) is always visible, never implicit — an analyst should never have to guess what a given run actually checked.

10 Design Decisions and Deviations from the Brief

The original build brief was used as a requirements baseline; the table below records where this implementation deviates from it and why, each decision made after auditing prior art rather than by default.

ID	Decision	Rationale
ADR-01	Kept the real-customer-data case study out of this repository	The Ebryx case report used to develop the manual methodology contains real customer headers and employee addresses; this repository is intended to be published, so it lives entirely separately.
ADR-02	Clean-room implementation, no third-party code reused	Four of eight reference projects audited grant no reuse rights at all; two more are GPL-3.0 despite GitHub or their own README misreporting the licence.
ADR-03	Implements live DNS/cryptographic verification	The differentiator; every reference project audited is parse-only.
ADR-04	No database, queue or cache server	Nothing needs to persist; avoids the dead-infrastructure failure mode observed in a reference project (PostgreSQL + Redis + Celery, none used meaningfully).
ADR-05	Offline by default, enrichment strictly opt-in	Analysts are frequently forbidden from submitting customer indicators to third parties; unique-per-recipient phishing URLs can tip off an attacker if submitted to a public scanner.
ADR-06	Server-rendered Jinja2, no client framework	Autoescaping is the primary XSS defence on hostile input; no build step, no CDN dependency.
ADR-07	Risk rules externalised to YAML with stable IDs	Lets the verdict cite exactly which rules fired, reviewable without reading Python.
ADR-08	<code>tldextract</code> with network fetch disabled, bundled PSL snapshot	A security tool making an unannounced outbound HTTP request during analysis is itself a finding.

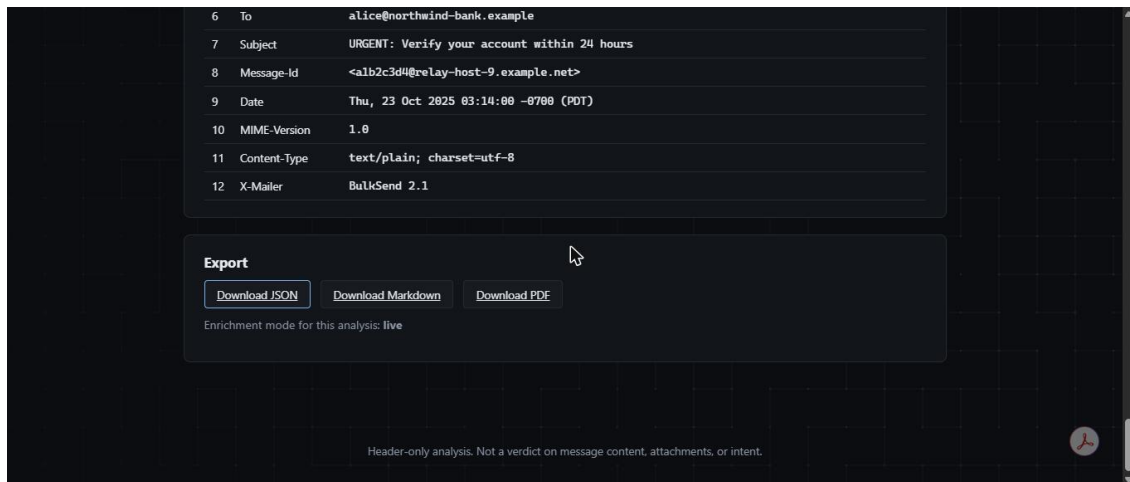


Figure 8: Export panel. Reports export as JSON (for tooling), Markdown (for a ticket or wiki page), or PDF — a standalone, print-formatted document generated server-side via a pure-Python HTML-to-PDF pipeline (`xhtml2pdf`), chosen specifically because it needs no external binary (no headless Chrome, no `wkhtmltopdf`, no GTK), so it installs cleanly cross-platform including Windows.

11 Threat Model and Limitations

11.1 What this tool is not

This is header-only analysis. It makes no claim about message body content, attachments, or sender intent — the footer on every results page says so explicitly. It cannot establish that a message is safe: a compromised-but-genuine account passes every check here. Independent verification checks *current* DNS state, which can differ from what was published at delivery time (a domain can rotate SPF/DKIM records after the fact).

11.2 Trust boundary

Authentication-Results headers are forgeable per RFC 8601 §7.1. Every asserted result defaults to UNKNOWN trust unless **TRUSTED_RECEIVER_DOMAINS** / **TRUSTED_RECEIVER_HOSTS** is explicitly configured to name infrastructure the operator actually controls. Leaving this unset is the safe default, not a bug.

11.3 Data handling

Raw headers are never placed in a URL or query parameter (asserted structurally by test — every route accepting header content is a POST-body route, checked by walking the FastAPI route table itself). **LOG_RAW_HEADERS** defaults to **false**. Reports live in a bounded, short-lived, in-memory cache with opaque IDs (`secrets.token_urlsafe`), never derived from message content.

12 Testing and Quality

253 tests pass with no network access and no API keys required for the suite itself (the live threat-intel path is exercised separately and manually, as in Figure 5, never as part of CI). The suite includes architectural boundary tests — e.g. asserting the core analysis modules perform no I/O

— and structural security tests, such as walking FastAPI's route table to assert no GET route can expose header content via a path parameter. Linting is enforced with **ruff**; every rule suppression in **pyproject.toml** is documented with a one-line rationale rather than left silent.

A test-isolation gap was found and fixed during this documentation pass: **Settings** reads **.env** from the working directory by pydantic-settings default, so a developer's local **.env** — enrichment keys, feature flags — silently leaked into any test that built a bare **Settings()** without explicitly overriding every affected field. Fixed with a session-scoped **conftest.py** fixture that disables the dotenv source for the whole test session, so test behaviour no longer depends on what happens to be sitting in the repo root.

13 Reference Repositories

Eight open-source projects were audited before implementation began, for licence terms and reusable concepts. Two licence traps were found and are worth recording: one project is reported as "Other" by GitHub's licence detector when it is actually GPL-3.0; another's README claims MIT while its actual LICENSE file is GPL-3.0. Four of the eight grant no reuse rights at all. The conclusion drawn — clean-room implementation throughout, no third-party code — is ADR-02 above. Full audit notes are in **docs/REFERENCE_REPOSITORIES.md** in the repository.

14 Setup and Deployment

- **Local run:** `pip install -r requirements.txt`, copy **.env.example** to **.env**, `uvicorn app.main:app`. Works fully offline with zero configuration — every provider reports **DISABLED** rather than inventing a result.
- **Container:** a production Dockerfile and **docker-compose.yml** are provided.
- **Secrets:** **.env** is git-ignored; nothing in it is ever committed. **CSRF_SECRET** should be generated per deployment (`python -c "import secrets; print(secrets.token_urlsafe(32))"`).

15 Conclusion

The header-only, independently-verified, evidence-cited approach documented here is a direct translation of a manual SOC workflow into software — not an approximation of one. Every design decision recorded in Section 9 traces back either to a gap found in existing open tooling or to a correctness bug the project's own test suite caught while building it. The result is a tool whose verdict an analyst can defend under questioning, because the tool itself can point at the exact line of evidence behind every claim it makes.